

Lecture 7 Task — Code You Never Write

Real-World AI Coding Projects

Objective

The purpose of this assignment is to help students practice the “Code You Never Write” approach — using AI to produce working code for real problems without writing, and often without even reading, the code themselves.

In this assignment the student is the **client**, not the programmer. Your job is to describe a real problem clearly, hand it to an AI tool, and then verify the result against facts you already know to be true. The goal is not to become a coder. The goal is to become a good client for code: explain the problem well, check the output carefully, and trust only what you can confirm.

Students will complete **four hands-on projects**. Each project uses the student’s own real data, so the final result is genuinely useful and not just a practice exercise.

About These Projects

These four projects are taken from the end of the **Code You Never Write** crash course (Panaversity Agent Factory). Each one solves a real-world problem that no ready-made app can solve, because the rules are unique to *you*. You will use any AI tool of your choice — such as ChatGPT, Claude, Gemini, or any AI coding assistant — to generate the code, and you will check every result against something you already know.

One rule applies to all four projects: never trust output you cannot check. Before you rely on any answer the AI gives you, confirm it against a number, total, or fact you already know is correct. If the AI’s result does not match your known fact, the code is wrong — not your fact.

Book & Reading (read before starting)

- **Book:** The AI Agent Factory — <https://agentfactory.panaversity.org/>
- **This lecture’s chapter:** Code You Never Write (crash course) — <https://agentfactory.panaversity.org/docs/code-you-never-write-crash-course>

Task 1 — Money Detective

Estimated time: about 45 minutes.

The Idea

“Don’t track your money going forward. Hunt the leak hiding in your own real history.” Instead of starting a new budget, you will examine your *past* transactions to find spending patterns, forgotten subscriptions, and duplicate charges — the kind of leaks ordinary apps miss because the rules for what counts as a “leak” are personal to you.

Requirements

- An export of your own real transaction history (for example bank or wallet statement) in a simple format containing date, description, and amount.
- At least one known baseline figure you can use to verify the result — for example your true total spend for a month, or a balance you already know.
- A rough idea of the spending categories that matter to you.

Steps

1. Paste your transaction data into the AI chat and ask it to write a script that finds recurring charges, possible forgotten subscriptions, and duplicate or repeated payments.
2. Verify the result by checking at least two figures by hand against totals you already know. If the numbers match your known totals, you can trust the rest.
3. Ask the AI to explain the logic of the code in plain English, so you understand what it is doing.
4. Save the final script so you can re-run it every month on fresh data.

Expected Outcome

A working script plus a concrete finding you can act on — for example, “I found a subscription I forgot to cancel” or “I was charged twice for the same thing.”

Task 2 — What’s My Grade, Really

Estimated time: about 30–45 minutes.

The Idea

“Encode the one set of grading rules no app has — your teacher’s — and find out where you really stand.” School grade apps do not know your specific teacher’s rules: dropped lowest scores, weighted categories, or conditional replacements. You will encode those exact rules and calculate your true current grade.

Requirements

- A complete list of your real scores (assignments, quizzes, midterms, exams).
- Your teacher’s actual grading policy, written out (weights per category, dropped scores, any special replacement or bonus rules).
- An understanding of how those rules interact with each other.

Steps

1. Paste your scores and your teacher’s grading rules into the AI as plain text and ask it to write a script that calculates your current grade.
2. Verify the output by calculating one category by hand and confirming it matches the script.
3. Ask the script to tell you the exact score you need on the final exam to reach a target grade.
4. Keep the script so you can update it with new scores throughout the term.

Expected Outcome

An accurate picture of where you actually stand, plus the specific final-exam score you need to hit the grade you are aiming for.

Task 3 — The Books Don't Match

Estimated time: about 45 minutes.

The Idea

“Two records that should agree — your counted total and the messy digital one. Find the gap, name the person.” You will reconcile a known, hand-counted total (such as trip dues, club funds, or money collected for a group gift) against a messy digital payment record with inconsistent or ambiguous memos.

Requirements

- A known correct total that you counted yourself.
- A digital payment record (for example, transfer history) with inconsistent or unclear formatting.
- Clear rules, based on your own knowledge, for interpreting ambiguous entries (for example, “a payment from this name really means this person”).

Steps

1. State your expected total up front so the AI knows the target the records should add up to.
2. Provide the payment records exactly as you received them, without cleaning them up first.
3. Document your personal rules for interpreting ambiguous entries and give them to the AI.
4. Ask the script to find the gap and identify the unmatched amounts and people that need follow-up.

Expected Outcome

A reconciled set of books: you know exactly which amounts are missing or unaccounted for, and who you still need to follow up with.

Task 4 — Organize the Mess (The Files You Forgot)

Estimated time: about 30–45 minutes.

The Idea

“Find what’s actually in your folders.” Most people accumulate digital clutter — duplicate files, forgotten downloads, screenshots, and documents saved in several places. Unlike the first three projects, this one involves **real file operations**, where a careless script could delete or rename the wrong files. This task teaches the safety discipline that goes with letting code touch your real files.

Requirements

- A genuinely messy folder on your computer (for example Downloads, Documents, or an old project folder).

- Real files you have forgotten about or are unsure whether to keep.

Steps (follow the safety order carefully)

1. **Copy first.** Before doing anything, make a backup copy of the entire folder. Always work on the copy until the code has earned your trust.
2. **Write the brief.** Describe in plain language what “clean” means for you — for example, find duplicates (same name or same contents), flag very large files, or group files by type.
3. **Demand a dry run.** Tell the AI explicitly: “Show me your full plan first — every file you would move, delete, or rename — and wait for my approval before touching anything.”
4. **Review the plan.** Read the full list of proposed operations before any code runs.
5. **Approve and execute.** Only after you approve the plan does the code run, and it should write results to new files or folders rather than overwriting the originals.
6. **Verify and keep.** Spot-check that the reorganized files are where you expect, then save the cleanup script for future use.

Expected Outcome

A clear view of what you actually own — for example, “I found 2 GB of duplicate photos I can delete” or “I rediscovered three half-finished documents worth returning to” — done safely, with your originals untouched.

Submission — One GitHub Repository for All Four Projects

For this lecture, every student must create **one single GitHub repository** that contains all four projects together. Do not create four separate repositories.

Recommended Repository Structure

- **README.md** (at the top level) — your name, a short summary of the four projects, and which AI tools you used.
- **project-1-money-detective/** — the script, a short README, the prompts you used, and screenshots of the result.
- **project-2-whats-my-grade/** — same contents for Task 2.
- **project-3-books-dont-match/** — same contents for Task 3.
- **project-4-organize-the-mess/** — same contents for Task 4.

Important: do not commit any private or sensitive personal data (real bank numbers, passwords, or anything you would not want public). Replace sensitive values with sample or dummy data in anything you upload, but keep enough to show the project works.

Each Project Folder Should Contain

- The final working script.
- A short README explaining the problem, the AI tool used, and how you verified the result.
- The prompts you used (initial prompt and any improved versions).
- Screenshots or a short clip showing the working result.

Deliverables

Each student must submit:

- The link to your single GitHub repository containing all four projects.
- A short report (Markdown in the repo, a PDF, or a Google Doc) covering all four projects.
- Screenshots or short demo clips for each project.

The Report Should Include (for each project)

- Project title and the problem it solves.
- Which AI tool(s) you used.
- The initial prompt and any improved prompts.
- How you verified the result against something you already knew.
- What worked, what did not, and any problems you faced.
- The final result and what you learned or were able to act on.

Final Submission Requirements

Submit all materials on the Panaversity Dashboard/Portal:

1. **GitHub Repository Link** — one repo with all four project folders.
2. **Report Document** — explaining the idea, the AI-assisted process, the prompts used, the verification step for each project, problems faced, and final results.
3. **Supporting Material** — screenshots, demo video, Google Docs link, PDF report, or any other useful files.

Submission Instructions

- Upload all deliverables to your Panaversity Dashboard/Portal.
- Make sure your GitHub repository is public (or accessible to the instructor) and all links work.
- Organize the repository with one clearly named folder per project.
- Remove or replace any private or sensitive personal data before uploading.
- Make sure each project has at least one working, verifiable result.

Evaluation Criteria

Students will be evaluated on:

- Use of the “client, not coder” approach — describing the problem clearly to the AI.
- Correct verification of each result against a known fact or total.
- Quality and clarity of the prompts and prompt iterations.
- Completeness — all four projects attempted and organized in one repository.
- Quality of documentation and the written report.

- Safe handling of file operations in Task 4 (working on a copy, demanding a dry run).
- Practical usefulness of the results on the student's own real data.
- Effort, experimentation, and overall completeness of the submission.